

OpenEASD

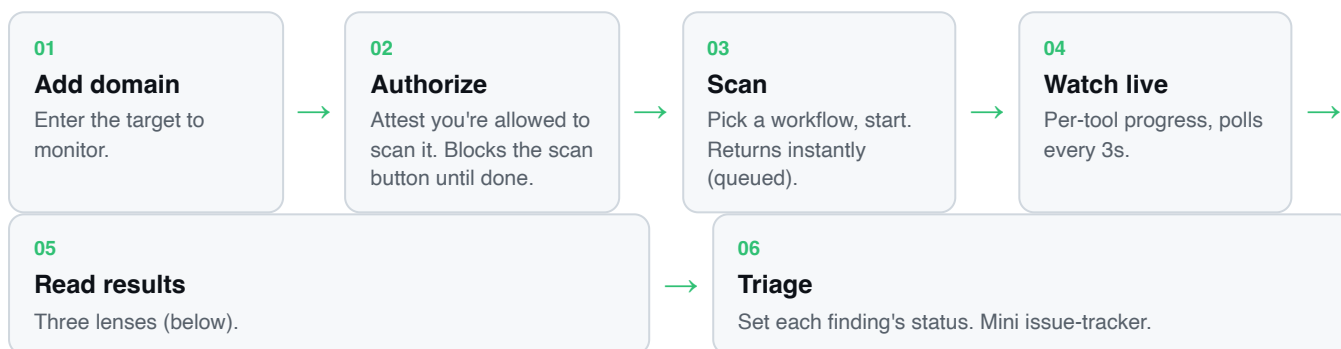
UX Flow Reference — for product design

External Attack Surface Detection platform · React 19 SPA over a Django API · v2.2.0 · Prepared for product-design review

What it is, in one line: a self-hosted security tool that scans a domain's public footprint for exposures and presents them as a living, browsable attack-surface inventory. The user is a **security operator** (single-user product — one admin, no team/RBAC).

1 · The core user journey

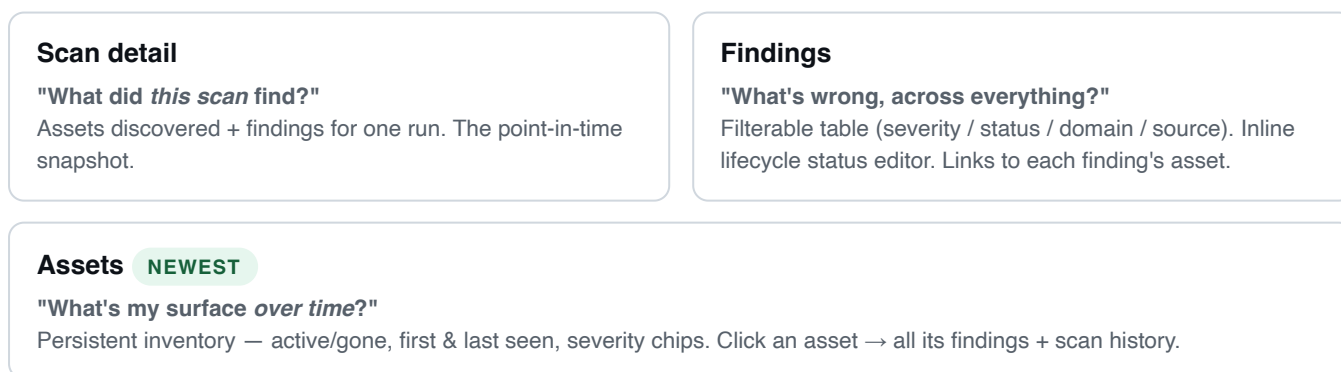
The everyday loop. Everything else in the product supports this spine.



DESIGN TENSION Step 02 (**Authorize**) is **deliberate friction** — the product refuses to scan a target the user hasn't attested they own or have consent for. This is a legal/ethical gate, not a UX smell. Any redesign must keep it unskippable, but the wording and moment could be made friendlier.

2 · The three result lenses

After a scan, the same data is read three ways. The pivot *between* them is the heart of the UX.



The **Asset** ↔ **Finding** pivot works both directions: from a finding jump to its asset; from an asset see every finding across every scan. This is the product's shift from *scan-centric* toward *asset-centric*.

3 · Screen inventory (the left-nav)

SCREEN	JOB TO BE DONE	PRIMARY INTERACTION
--------	----------------	---------------------

Dashboard	At-a-glance state — KPIs, per-domain status, urgent findings, asset inventory count	Read + drill-down
Domains	Manage targets + authorization + monitoring cadence	Add / toggle / authorize
Scans	Start scans, watch live progress, browse history	Start + live poll
Assets	The persistent attack-surface inventory (browse / filter / pivot)	Filter, search, click-through
Findings	Work the issues — filter + set lifecycle status	Filter + inline edit
Workflows	Power-user: customise which tools run	Toggle tools
Insights	Trend view — Exposure Score + grade, are things improving?	Read (charts)
Notifications	Configure Slack/Teams alerts, view history	Configure once
AI Analysis	Optional, opt-in: "Fix these first" triage ranking	Enable (consent) + read

4 · Two usage modes

The same product serves two very different rhythms — worth designing for both explicitly.

① Active / interactive

Log in → add → authorize → scan → triage. Hands-on, session-based. The "I'm investigating right now" mode.

② Passive / monitoring

Set a domain to auto-rescan (6h / daily / weekly). Scans run in the background; a Slack/Teams alert pulls the user in only when the surface changes. The "set & forget, notify me" mode.

Mode ② means many users' primary entry point is **an alert notification**, not the dashboard — the deep-link target (a scan result or a changed asset) deserves as much design care as the home screen.

5 · UX design principles (what shapes the feel)

Consent-first. You cannot scan without attesting authorization. Friction by design — it's a security tool, and scanning without permission is a real harm.

Asset-centric. Findings are transient; the **asset inventory persists**. "Do I still have that forgotten host exposed?" is the durable question the product is evolving to answer first.

Live + durable. 3-second polling makes long (minutes-long) scans feel responsive; the engine resumes crashed scans, so a scan never silently dies mid-run.

Honest scope. The product distinguishes "not found" from "couldn't check" (blocked by a WAF, partial scan). The UI should never imply coverage it didn't achieve.

6 · Visual language (current)

Dark theme throughout. Tokens below are the live values — a starting palette to work from.



#0d1117

bg



#161b22

card



#30363d

border



#30c074

accent



#f85149

critical



#d29922

warn

Severity is colour-coded consistently (critical → info) via a shared **Badge** component; status pills reuse the same system. Typography: system sans for UI, monospace for hosts/IPs/hashes/keys.

7 · Open questions for design

Where a designer's eye would add the most value:

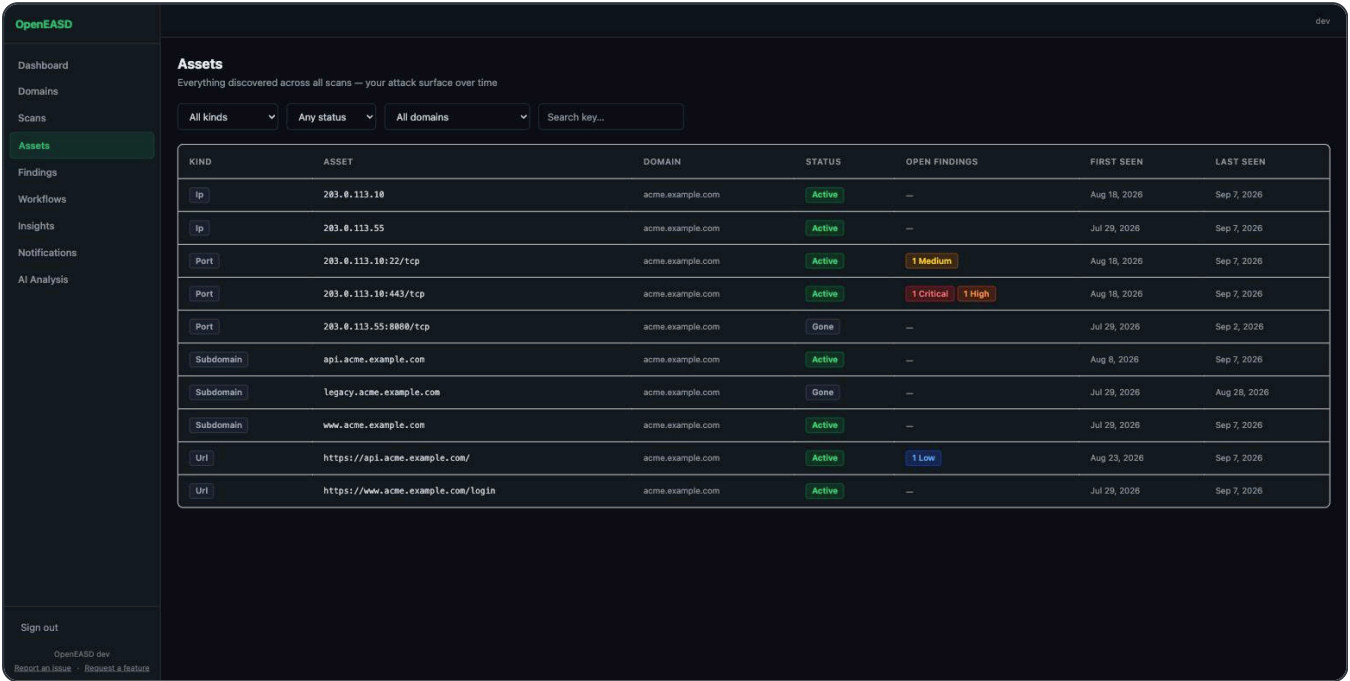
AREA

THE OPPORTUNITY

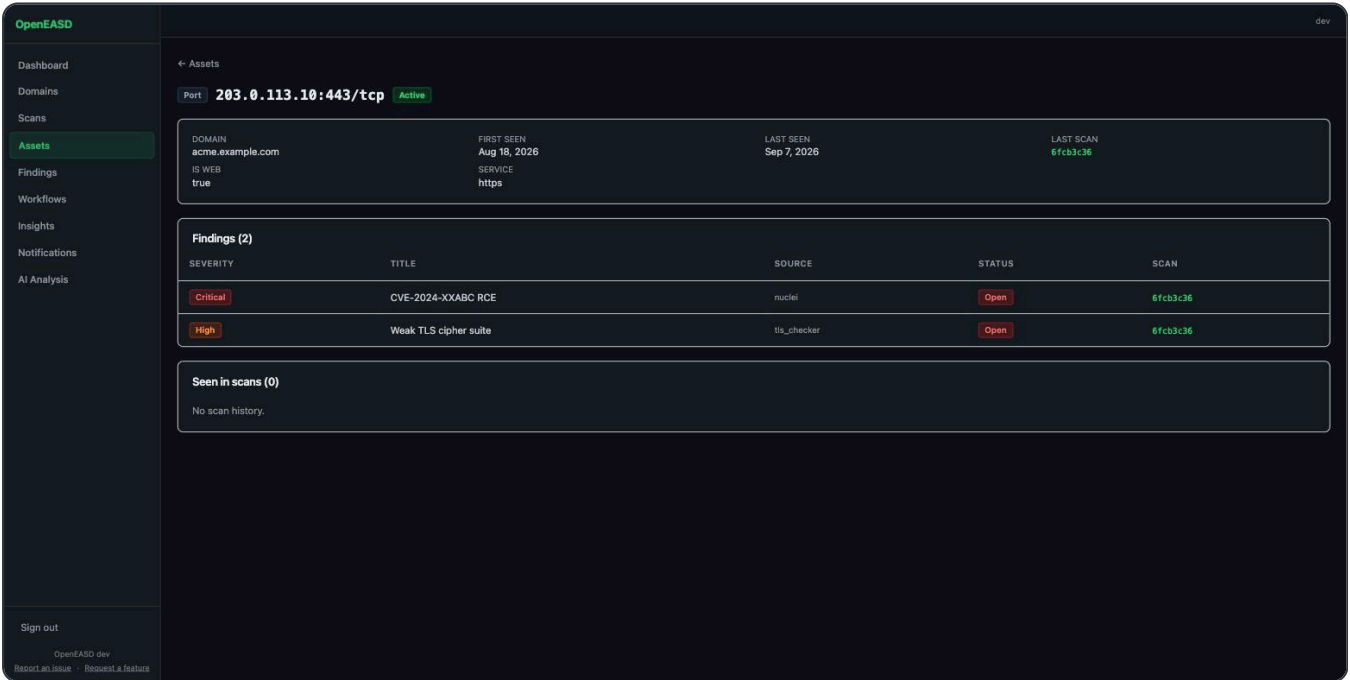
Onboarding	First-run is bare (login → forced password change → empty dashboard). The add→authorize→scan path could be a guided first-time flow.
The authorize moment	Legally required friction, but the copy + timing feel like a gate. Can it feel like a confident, reassuring step instead of a blocker?
Asset-centric shift	The product is mid-transition from scan-centric to asset-centric. Should Assets become the primary landing view rather than Dashboard/Scans?
Alert deep-links	Monitoring users arrive from a Slack alert. That landing (the changed asset / new finding) is under-designed vs. the home screen.
Findings triage	The lifecycle editor is a dropdown per row. At scale, is there a better bulk / keyboard / kanban model?
Severity density	A scan can produce dozens–hundreds of findings. The "which 3 matter" story (today: optional AI + Exposure Score) could be a stronger default UX, not a side feature.

8 · Current screens (captured live)

The asset-centric views as they render today (v2.2.0) — the newest part of the product and the direction it's moving. Shown for reference, not as a design target.



Assets — the inventory list. Every subdomain / IP / port / URL the domain has exposed, with **Active / Gone** status, first & last seen, and per-asset severity chips. Filter by kind/status/domain + free-text search. Each row clicks through to detail.



Asset detail — the pivot. One asset's metadata + its findings across scans (each linking back to the scan) and a "seen in scans" timeline. This is where the surface-over-time story pays off.

Screens captured from a live instance with seeded sample data. Dark theme is the live default; these are the real components, not mockups.

The one-sentence UX arc: Add a domain → attest you may scan it → run a scan (watch it live) → read results three ways (this scan / all findings / the asset inventory over time) → triage each finding — with optional summaries (AI),

scoring (Insights), exports (Reports), and push alerts (Notifications) around the loop.

This document describes the current (v2.2.0) experience as a baseline for design work. It is a UX reference, not a spec — screens and flows are the live product; the "open questions" are invitations, not requirements.